Inventory Backend

Reference for Frontend & Integration Teams

Phases 1–8 complete · 36 endpoints · 10 tables · 199 tests

Inventory Backend — Reference

Audience: Frontend developers, QA, integration teams, future maintainers. **Scope:** Everything you need to consume this API correctly, plus the engineering decisions behind it so you can predict its behaviour.

> Generated from the live codebase. The OpenAPI explorer at `http://localhost:8000/docs` is the canonical, always-current contract.

---

1. What this service is

The Inventory Backend is the **source of truth** for:

- Item catalog (raw materials + finished goods, with stock levels)
- Customer and vendor master data
- Vendor-item pricing terms (date-windowed, per-item rates with discount)
- Purchase orders (buys — stock IN)
- Sales orders (sells — stock OUT)
- An **append-only audit ledger** for every stock change

It does **not** call Zoho CRM directly. CRM sync is the Integration Layer's job (Phase 9, deferred). Frontend talks to this service over HTTPS with a bearer JWT.

---

2. Architecture

```
React Frontend ■ HTTPS, Bearer JWT ▼ FastAPI Backend ■■■■■■ PostgreSQL (inventory_db) ■ internal HTTP (Phase 9) ▼ Integration Layer ■■■■ Zoho CRM
```

Inside the service, one-way dependency:

```
api/ → services/ → repositories/ → models/ ▲ schemas/, dependencies/
```

Layer	Owns	Forbidden
`api/`	Routing, request/response shape, status codes	SQL, business rules
`schemas/`	Pydantic models for I/O validation	DB models, business logic
`services/`	Business rules, multi-step orchestration	Raw SQL, FastAPI imports
`repositories/`	All DB queries (SQLAlchemy)	HTTP concerns, business rules
`models/`	ORM table definitions	Logic beyond hybrid/computed columns

A route handler is typically ≤ 10 lines: **parse** → **call service** → **return**.

---

3. Engineering principles followed

These are the rules the codebase is held to. They show up across every phase — so when you read the code or build a feature on top, you can predict behaviour.

Principle	What it means in this codebase
Spec-driven + TDD	Every phase is written as failing tests first, then code, then refactor under green. 199 tests pin the contract.
Single source of truth	The DB is authoritative; computed fields (e.g. an item's `status`) are never stored. Stock is mutated by exactly one function (`record_movement`).
DB-level invariants	Critical rules (stock ≥ 0, quantity > 0, status↔date consistency) are enforced by `CHECK` constraints and `UNIQUE` indexes — not just by Pydantic. The DB is the contract; the schema is defence-in-depth.
Strict typing	mypy strict on `src/`. Every public function fully annotated.
Append-only audit	`stock_movements` is insert-only at every layer (model, repository, service, API). No PATCH, no DELETE — ever.
Caller owns transactions	Services that compose other services (PO `receive`, SO `ship`) do not `commit()` inside `record_movement`. The outer service commits once after all lines apply, so a mid-loop failure rolls back everything.
Deterministic locking	When a flow touches multiple item rows, they're locked in `item_id`-sorted order, so concurrent POs/SOs cannot deadlock.
Soft-delete only	`is_active=false` everywhere; nothing is hard-deleted. Preserves audit trails (`created_by_user_id` FKs use `ON DELETE RESTRICT`).
Refresh after commit	Per `CLAUDE.md §10.1` — every commit on a row with a server-onupdate column is followed by `await session.refresh(...)` to avoid the `MissingGreenlet` trap.
No speculative code	Three similar lines is fine; three similar functions is a smell. No "future-proof" knobs.

Error envelope is stable	Every non-2xx body has `{error: {code, message, request_id}}`. Clients branch on `code`.
No secrets in logs	Passwords, tokens, full Authorization headers are never logged.

---

4. Authentication & authorization

Auth scheme: OAuth 2.0 Bearer (JWT, HS256). Tokens carry only `sub=user_id`; every authenticated request does a fresh user lookup so revocation (deactivation) takes effect immediately.

Token lifetime: 8 hours by default (configurable via `ACCESS_TOKEN_EXPIRE_MINUTES`). There is **no refresh endpoint** in Phase 1 — re-login is the only renewal path, and the long lifetime keeps it rare. The `expires_in` field on the login response lets the Frontend warn the user before the session lapses and auto-redirect to login on `401`. A proper OAuth2 refresh-token flow (rotating refresh tokens, server-side revocation list) is a Phase 2+ build, not a half-version bolted on now.

Password hashing: `pwdlib` with Argon2id (not bcrypt). One-time choice documented in `CLAUDE.md`.

Headers:

> `Authorization: Bearer <jwt> X-Request-ID: <uuid> # added by the server on the response; clients may # send one to be echoed back for correlation`

Roles in Phase 1: A single `is_admin` boolean on `users`. There is no RBAC framework yet — admin operations are gated by a `require_admin` dependency. There is **no admin-promotion endpoint** by design (avoids the "first to register becomes god" race); admins are made via SQL after registration. See README → Administration.

Admin-only endpoints: - `GET /api/v1/users` - `DELETE /api/v1/users/{user_id}` - `DELETE /api/v1/customers/{customer_id}` - `DELETE /api/v1/vendors/{vendor_id}`

Auth failure codes:

Code	HTTP	Meaning
`MISSING_TOKEN`	401	No bearer header sent
`INVALID_TOKEN`	401	JWT failed signature/decode
`EXPIRED_TOKEN`	401	JWT past `exp`
`INVALID_CREDENTIALS`	401	Login: bad email *or* password (one code for both to prevent enumeration)
`ADMIN_REQUIRED`	403	Authenticated but not admin

---

5. Error model

```
{ "error": { "code": "INSUFFICIENT_STOCK", "message": "Insufficient stock: 5 available, 50 requested", "request_id": "f04ca71a-602e-4f78-8fcf-c2f2dc315170" } }
```

- `code` is the stable identifier — branch on this, not on `message`.
- `message` is human-readable; subject to change without notice.
- `request_id` matches the `X-Request-ID` response header.

HTTP status conventions

Status	When
200	OK on GET / action endpoints (`/receive`, `/ship`)
201	Resource created
204	Soft-delete succeeded (no body)
401	Authentication problem
403	Authorization problem (you're known but not allowed)
404	Resource doesn't exist OR you're not allowed to see it (to prevent enumeration)
409	State conflict — duplicate key, idempotency violation, insufficient stock
422	Pydantic schema validation OR a domain validation error (e.g. `PRICE_UNAVAILABLE`)
500	Unexpected — should never happen in normal operation

Stable error codes (alphabetical)

Code	HTTP	When raised
`ADMIN_REQUIRED`	403	Non-admin called an admin-only route
`CANNOT_DEACTIVATE_SELF`	409	Admin tried to deactivate their own account
`CUSTOMER_NOT_FOUND`	404	Unknown or inactive customer
`DUPLICATE_CUSTOMER`	409	DB uniqueness collision on customer create/update
`DUPLICATE_LINE_ITEM`	409	Same `item_id` listed twice on a PO/SO payload

`DUPLICATE_PURCHASE_ORDER`	409	DB constraint hit while creating a PO (defensive)
`DUPLICATE_SALES_ORDER`	409	DB constraint hit while creating an SO (defensive)
`DUPLICATE_SKU`	409	SKU already taken
`DUPLICATE_TERM`	409	DB uniqueness on vendor-term update
`DUPLICATE_VENDOR`	409	DB uniqueness collision on vendor create/update
`EMAIL_ALREADY_REGISTERED`	409	`POST /auth/register`
`EXPIRED_TOKEN`	401	JWT past `exp`
`INSUFFICIENT_STOCK`	409	OUT line would push stock below zero
`INVALID_CREDENTIALS`	401	Login failed (same code for bad email and bad password)
`INVALID_DATE_RANGE`	422	Vendor term update would produce `effective_to < effective_from`
`INVALID_REFERENCE_PAIR`	409	Internal: `reference_type` and `reference_id` not both set or both NULL
`INVALID_TOKEN`	401	JWT failed signature/decode
`ITEM_NOT_FOUND`	404	Unknown or inactive item
`MISSING_TOKEN`	401	No `Authorization` header
`OVERLAPPING_TERMS`	409	New vendor term overlaps an active one
`PO_NOT_DRAFT`	409	Receive attempted on a non-DRAFT PO (idempotency)
`PRICE_UNAVAILABLE`	422	PO line has no explicit `unit_price` and no active vendor term
`PURCHASE_ORDER_NOT_FOUND`	404	Unknown PO id
`SALES_ORDER_NOT_FOUND`	404	Unknown SO id

`SO_NOT_DRAFT`	409	Ship attempted on a non-DRAFT SO (idempotency)
`TERM_NOT_FOUND`	404	Unknown term OR belongs to a different vendor
`USER_NOT_FOUND`	404	Unknown user OR caller has no right to see it
`VENDOR_NOT_FOUND`	404	Unknown or inactive vendor

---

6. Pagination

Every list endpoint returns the same envelope:

```
{ "items": [ ... ], "total": 142, "limit": 25, "offset": 0 }
```

- `total` is the count **after filters but before pagination** — what the UI needs to render pager controls.
- `limit` constraint: `1 ≤ limit ≤ 100` (default 25). Server enforces.
- `offset` constraint: `offset ≥ 0` (default 0).

---

7. The data model

Entity-relationship summary

```
users ■■■■■■ items (audit FKs) ■■■■ customers ■■■■ vendors ■■■■ vendor_item_terms ■■■■
stock_movements ■■■■ purchase_orders ■■■■ sales_orders customers ■■■■■■■■ sales_orders
■■■■■■ sales_order_items ■■■■■■ items vendors ■■■■■■■■ purchase_orders■■■■■
purchase_order_items ■■■■ items items ■■■■■■■■ stock_movements items ■■■■■■■■
vendor_item_terms ■■■■■■■■ vendors (soft-FK polymorphic):
stock_movements.(reference_type, reference_id) → purchase_orders.id when reference_type =
'PURCHASE_ORDER' sales_orders.id when reference_type = 'SALES_ORDER'
```

Tables

Table	Phase	Key columns
`users`	2	`id`, `email` (unique), `password_hash`, `full_name`, `is_admin`, `is_active`, audit timestamps

items	4	id, sku (unique), name, type (RAW/FINISHED), category, unit_of_measure, stock_quantity Numeric(20,4), reorder_threshold, unit_price, is_active, audit (user FKs + timestamps)
customers	5	id, company_name, contact_person, email, phone, customer_code (unique-when-set), gstin (unique-when-set), is_privileged, is_active, audit
vendors	5	id, vendor_name, contact_person, email, phone, vendor_code, gstin, is_active, audit
vendor_item_terms	5	id, vendor_id FK, item_id FK, rate Numeric(14,2), discount_percent Numeric(5,2), effective_from, effective_to?, is_active, audit
stock_movements	6	id, item_id FK, direction (IN/OUT), reason (PURCHASE/SALE/ADJUSTMENT), quantity Numeric(14,3), stock_before, stock_after, reference_type?, reference_id?, remarks?, created_by_user_id, created_at
purchase_orders	7	id, po_number (unique, PO-YYYYMM-NNNNNN), vendor_id FK, order_date, expected_delivery_date?, received_date?, status (DRAFT/RECEIVED), subtotal, total, notes?, audit
purchase_order_items	7	id, purchase_order_id FK CASCADE, item_id FK, quantity, unit_price, line_total, created_at. UNIQUE (purchase_order_id, item_id)

`sales_orders`	8	`id`, `so_number` (unique, `SO-YYYYMM-NNNNNN`), `customer_id` FK, `order_date`, `expected_delivery_date?`, `shipped_date?`, `status` (DRAFT/SHIPPED), `subtotal`, `total`, `notes?`, audit
`sales_order_items`	8	`id`, `sales_order_id` FK CASCADE, `item_id` FK, `quantity`, `unit_price`, `line_total`, `created_at`. UNIQUE (sales_order_id, item_id)

DB invariants (CHECK constraints)

These are the rules the database enforces independently of application code:

- `items.stock_quantity >= 0`
- `items.unit_price >= 0`
- `items.reorder_threshold IS NULL OR reorder_threshold >= 0`
- `stock_movements.quantity > 0` (direction carries the sign)
- `stock_movements.stock_after >= 0` (the ultimate stock-non-negative backstop)
- `stock_movements`: reference pair consistency — both fields NULL or both NOT NULL
- `vendor_item_terms.rate >= 0`
- `vendor_item_terms.discount_percent BETWEEN 0 AND 100`
- `vendor_item_terms.effective_to IS NULL OR effective_to >= effective_from`
- `purchase_orders.subtotal >= 0`, `total >= 0`
- `purchase_orders`: status↔received_date consistency (`RECEIVED ⇔ received_date IS NOT NULL`)
- `purchase_order_items.quantity > 0`, `unit_price >= 0`, `line_total >= 0`
- `sales_orders.subtotal >= 0`, `total >= 0`
- `sales_orders`: status↔shipped_date consistency (`SHIPPED ⇔ shipped_date IS NOT NULL`)
- `sales_order_items.quantity > 0`, `unit_price >= 0`, `line_total >= 0`

---

8. The keystone — how stock changes

`items.stock_quantity` is mutated by exactly one function in the codebase: `StockMovementService.record_movement`. It:

1. `SELECT ... FOR UPDATE`s the item row (serialises concurrent decrements).
2. Computes `stock_after = stock_before ± quantity`.
3. Rejects OUT with `stock_after < 0` → 409 `INSUFFICIENT_STOCK`.
4. Updates `items.stock_quantity` AND inserts a `stock_movements` row in the **same session**.
5. Does **not** commit — the caller owns the transaction.

Three flows call `record_movement`:

Flow	Direction	Reason	reference_type
Manual adjustment (`POST /stock-movements/adjustments`)	IN or OUT	`ADJUSTMENT`	`NULL` (with required `remarks`)
PO receive (`POST /purchase-orders/{id}/receive`)	IN	`PURCHASE`	`'PURCHASE_ORDER'`
SO ship (`POST /sales-orders/{id}/ship`)	OUT	`SALE`	`'SALES_ORDER'`

Each PO/SO action calls `record_movement` per line inside one outer transaction. If line N fails (e.g. `INSUFFICIENT_STOCK` on an SO), the service explicitly rolls back, so lines 1…N-1 are reverted too. **Stock moves completely or not at all.**

---

9. Endpoint reference

All paths are prefixed with `/api/v1` unless noted. `/health` is the only unversioned endpoint.

Every authenticated endpoint requires `Authorization: Bearer <token>`. The collection-level error envelope applies to every non-2xx response.

9.1 Auth

`POST /auth/register`

Register a new user.

Body

```
{ "email": "operator@example.com", "full_name": "Test Operator", "password": "correct-horse-battery-staple" }
```

Returns `201 { id, email, full_name, is_admin, is_active, created_at, updated_at }` **Errors** `409 EMAIL_ALREADY_REGISTERED`, `422`

`POST /auth/login`

Exchange email + password for a JWT.

Body

```
{ "email": "operator@example.com", "password": "..." }
```

Returns `200 { access_token, token_type: "bearer", expires_in, user: { ... } }` **Errors** `401 INVALID_CREDENTIALS`

`expires_in` is in seconds (default 28 800 = 8 hours). When this lapses, the client must redirect to login — there is no `/auth/refresh`.

9.2 Users

`GET /users` *(admin only)*

List users (paginated).

Query `limit, offset` **Errors** `401 MISSING_TOKEN`, `403 ADMIN_REQUIRED`

`GET /users/{user_id}`

One user. Non-admin may only fetch their own id; others return `404 USER_NOT_FOUND` (id-enumeration guard).

`PATCH /users/{user_id}`

Partial update.

- Self: `full_name` only
- Admin: `full_name`, `is_active`, `is_admin`

Errors `403 ADMIN_REQUIRED` (when non-admin sets is_active/is_admin), `404 USER_NOT_FOUND`

`DELETE /users/{user_id}` *(admin only)*

Soft-deactivate (`is_active=false`). Idempotent. `204`. **Errors** `404 USER_NOT_FOUND`, `409 CANNOT_DEACTIVATE_SELF`

9.3 Items

`POST /items`

Create an item.

Body

```
{ "sku": "BOT-1L-001", "name": "1L Bottle (Round)", "type": "FINISHED", "category": "Bottle", "unit_of_measure": "pcs", "unit_price": "45.00", "stock_quantity": "0", "reorder_threshold": "100" }
```

Returns `201` — **minimal envelope** `{ id, sku, name, created_at }`. Use `GET /items/{id}` for the full record. **Errors** `409 DUPLICATE_SKU`, `422`

`GET /items`

Query `limit, offset, type (RAW|FINISHED), category, search` Search is case-insensitive over SKU + name.

`GET /items/{item_id}`

Returns the full record, including computed `status` (`IN_STOCK | LOW_STOCK | NO_STOCK`).

`PATCH /items/{item_id}`

Partial update. **`sku`**, **`type`**, and **`stock_quantity`** are not patchable — stock changes go through the movement ledger.

9.4 Customers

`POST | GET (list) | GET /{id} | PATCH /{id} | DELETE /{id}` *(DELETE is admin-only soft-delete)*

`POST /customers`:

```
{ "company_name": "Acme Distributors Pvt Ltd", "contact_person": "Anita Sharma", "phone":
"+91 9876543210", "email": "ops@acme.example.com", "is_privileged": false,
"customer_code": "ACME-001" }
```

`is_privileged` is a flag reserved for future per-customer pricing; currently informational.

List filters: `search` (company_name + customer_code), `is_privileged`, `is_active`.

9.5 Vendors

`POST | GET (list) | GET /{id} | PATCH /{id} | DELETE /{id}` *(DELETE admin only)*

Same shape as customers minus `is_privileged`. **List filters:** `search`, `is_active`.

9.6 Vendor terms

Nested under a vendor — full CRUD.

`POST /vendors/{vendor_id}/terms`

```
{ "item_id": "...", "rate": "150.00", "discount_percent": "5.00", "effective_from":
"2026-01-01", "effective_to": "2026-12-31" }
```

Returns `201` **Errors** `404 VENDOR_NOT_FOUND/ITEM_NOT_FOUND`, `409 OVERLAPPING_TERMS`

Two active terms for the same (vendor, item) may not have overlapping date ranges. `effective_to` may be `null` (open-ended).

`GET /vendors/{vendor_id}/terms`

Query `limit, offset, item_id, active_only`

`GET /vendors/{vendor_id}/terms/{term_id}`

Returns `404 TERM_NOT_FOUND` when the term belongs to a different vendor (cross-vendor probing guard).

`PATCH /vendors/{vendor_id}/terms/{term_id}`

Re-checks overlap on the post-update range.

`DELETE /vendors/{vendor_id}/terms/{term_id}`

Soft-delete (`is_active=false`). Idempotent. `204`. Any authenticated user (pricing isn't an admin-only boundary).

9.7 Stock movements

The append-only audit ledger. No PATCH, no DELETE — ever.

`GET /stock-movements`

Query `limit, offset, item_id, direction (IN|OUT), reason (PURCHASE|SALE|ADJUSTMENT), date_from, date_to`

`date_from`/`date_to` filter on `created_at` inclusive on both ends.

Each row includes a computed `signed_quantity` (`+` for IN, `-` for OUT) for UI convenience.

`POST /stock-movements/adjustments`

Manual adjustment — the only direct write to the ledger from the API.

```
{ "item_id": "...", "direction": "IN", "quantity": "25", "remarks": "Recount correction - line B" }
```

`remarks` is required — accountability gate for a write that has no other paper trail. **Errors** `404 ITEM_NOT_FOUND`, `409 INSUFFICIENT_STOCK` (OUT below zero), `422` (missing/empty remarks etc.)

9.8 Purchase Orders (Phase 7)

State machine: `DRAFT → RECEIVED` (terminal). No PATCH, no DELETE.

`POST /purchase-orders`

Create DRAFT.

```
{ "vendor_id": "...", "expected_delivery_date": "2026-07-15", "notes": "Initial stock-up", "items": [ { "item_id": "...", "quantity": "100", "unit_price": "45.00" }, { "item_id": "...", "quantity": "50" } ] }
```

Pricing fallback: if `unit_price` is omitted on a line, the service looks up the most recent active `vendor_item_term` for `(vendor, item)` and uses `rate * (1 – discount_percent/100)` quantized to 2 dp HALF_UP.

Errors `404 VENDOR_NOT_FOUND/ITEM_NOT_FOUND` (also inactive), `409 DUPLICATE_LINE_ITEM`, `422 PRICE_UNAVAILABLE`

`GET /purchase-orders`

Query `limit, offset, vendor_id, status (DRAFT|RECEIVED), date_from, date_to`

`GET /purchase-orders/{po_id}`

Returns full PO including all lines.

`POST /purchase-orders/{po_id}/receive`

The keystone. Atomically:

1. `SELECT FOR UPDATE` the PO.
2. For each line in `item_id`-sorted order: `record_movement(IN, PURCHASE, reference_type='PURCHASE_ORDER', reference_id=po.id)`.
3. Sets `status=RECEIVED, received_date=today`.
4. Single commit.

Errors `404 PURCHASE_ORDER_NOT_FOUND`, `409 PO_NOT_DRAFT` (state-boundary idempotency).

9.9 Sales Orders (Phase 8)

State machine: `DRAFT → SHIPPED` (terminal). No PATCH, no DELETE.

`POST /sales-orders`

Create DRAFT.

```
{ "customer_id": "...", "expected_delivery_date": "2026-07-20", "notes": "Trial dispatch",
"items": [ { "item_id": "...", "quantity": "5", "unit_price": "60.00" }, { "item_id":
"...", "quantity": "10" } ] }
```

Pricing fallback: if `unit_price` is omitted on a line, the service uses the item's catalog `unit_price` (Phase 1 has no per-customer pricing table).

No stock pre-check at create time — operators may book a DRAFT SO knowing stock will be procured before shipping. Stock is enforced at `/ship`.

Errors `404 CUSTOMER_NOT_FOUND/ITEM_NOT_FOUND` (also inactive), `409 DUPLICATE_LINE_ITEM`

`GET /sales-orders`

Query `limit, offset, customer_id, status (DRAFT|SHIPPED), date_from, date_to`

`GET /sales-orders/{so_id}`

`POST /sales-orders/{so_id}/ship`

The OUT keystone — and the moment Phase 6's `SELECT FOR UPDATE` earns its keep. Atomically:

1. `SELECT FOR UPDATE` the SO.
2. For each line in `item_id`-sorted order: `record_movement(OUT, SALE, reference_type='SALES_ORDER', reference_id=so.id)`.
3. Sets `status=SHIPPED, shipped_date=today`.
4. Single commit.

Failure modes: - `404 SALES_ORDER_NOT_FOUND` - `409 SO_NOT_DRAFT` (already shipped — state-boundary idempotency) - `409 INSUFFICIENT_STOCK` — any line would push stock below zero. **Entire ship is rolled back.** No partial stock, no partial ledger, SO stays DRAFT and can be retried after stock arrives.

9.10 Probes

`GET /health`

Unversioned. Liveness — `200 {"status":"ok"}` if the process is up. Does **not** check downstream dependencies. Wire this to k8s `livenessProbe`.

`GET /ready`

Unversioned. Readiness — pings the database with `SELECT 1` under a 1-second timeout.

- `200 {"status":"ready"}` if the DB is reachable.
- `503` with `{error: {code: "DB_UNAVAILABLE", ...}}` if the ping fails (timeout, connection refused, auth, etc.).

Wire this to k8s `readinessProbe` so traffic routes away from a pod whose DB is down.

---

10. Recommended Frontend integration patterns

10.1 Auth flow

1. `POST /auth/register` (operator self-serve OR admin pre-provisions).
2. `POST /auth/login` → store `access_token`, `expires_in`, and the included `user` payload (so the UI doesn't need an immediate follow-up `GET /users/{id}` to learn its own identity / admin flag).
3. Compute the session deadline once: `expiresAt = Date.now() + expires_in * 1000`. Stash alongside the token.
4. Attach `Authorization: Bearer <token>` to every subsequent request.
5. About 1 minute before `expiresAt`, show a non-blocking "Session ends soon — please save your work" banner.
6. On any `401` (or once `Date.now() > expiresAt`) → clear the token and route to the login screen. The same code (`INVALID_CREDENTIALS`) covers both bad email and bad password on the login attempt — surface a single, deliberately vague message.

There is no `/auth/refresh`. Re-login is the only renewal path. The 8-hour default lifetime is long enough that this is a once-per-shift event for an operator, not a UX cliff. If a future iteration needs shorter sessions (RBAC, multi-device), we'll add a real OAuth2 refresh-token flow at that time.

10.2 Listing screens

- Always pass `limit` (≤ 100). Default 25 is reasonable.
- Use `total` for pager UI; clamp the requested `offset` so `offset + limit ≤ total`.
- For search UIs, debounce the `search` param ~250 ms — server does case-insensitive `LIKE`.

10.3 Editing screens (PATCH endpoints)

- Send **only the fields the user actually changed**. Pydantic `extra="forbid"` will return 422 on typos — useful, but also means sending the entire form back is risky (a field the API doesn't accept will reject the whole request).
- `PATCH /items/{id}` rejects `sku`, `type`, `stock_quantity` — handle those in code paths that don't use the form (stock = movement endpoints; SKU is identity).

10.4 Decimal handling

Money and quantities are JSON strings, not numbers, to preserve precision:

- `stock_quantity`: `Numeric(20, 4)` → up to 4 decimals.
- `quantity` on movements/PO/SO lines: `Numeric(14, 3)` → 3 decimals.
- Money columns (`unit_price`, `line_total`, `subtotal`, `total`): `Numeric(14, 2)`.

On the client: parse with `BigNumber.js` / `decimal.js`. **Never** use JavaScript `parseFloat` on these — you'll lose precision the moment you do arithmetic.

10.5 Receive / Ship UX

Both actions are POSTs with no body. On success:

- Refresh the PO/SO view (status flipped, `received_date`/`shipped_date` populated).
- Refresh the item screens for affected items (stock changed) — or rely on a global "stock changed" event.

- Show the corresponding ledger row by filtering `GET /stock-movements?item_id=...` or `reference_id=<po_or_so_id>` (filter parameter `reference_id` is not yet exposed in v1; for now filter client-side or query by `item_id`).

On `409 INSUFFICIENT_STOCK` for a ship: surface which items lack stock by re-fetching the SO + the items, or by inspecting the error `message` (currently includes the offending item's `before` and `requested` values).

10.6 Audit trail UI

The ledger is the system's append-only memory. Every quantity change is visible at `GET /stock-movements?item_id=<id>` with `direction`, `reason`, the linking `reference_type`/`reference_id`, and human-readable `remarks`.

For PO/SO detail views, render movements where `reference_id == po.id` (or `so.id`) — gives the operator a one-glance "what did receiving this PO do to my stock?" view.

---

11. Versioning & compatibility

- All resource endpoints are under `/api/v1`. A breaking change goes to `/api/v2`; `/api/v1` does not mutate.
- Adding new fields to existing responses is **not** a breaking change — clients must tolerate unknown fields.
- Adding new error codes inside an existing status code class is **not** a breaking change — clients must tolerate unknown codes.
- Renaming or removing an error code is breaking; we'll bump the API version if that ever happens.

---

12. Local development for Frontend devs

A minimal repro to get a working Backend in front of you:

```
# from Code/ root docker compose up -d # boots Postgres # from Backend/ copy .env.example .env # then edit JWT_SECRET_KEY uv sync uv run alembic upgrade head uv run uvicorn app.main:app --reload --app-dir src --port 8000
```

Open http://localhost:8000/docs for the live OpenAPI explorer (swagger).

Quick end-to-end:

```
# 1. Register curl -X POST http://localhost:8000/api/v1/auth/register \ -H "Content-Type: application/json" \ -d '{"email":"op@example.com","full_name":"Op","password":"correct-horse-battery-staple"}' # 2. Login → grab access_token curl -X POST http://localhost:8000/api/v1/auth/login \ -H "Content-Type: application/json" \ -d '{"email":"op@example.com","password":"correct-horse-battery-staple"}' # 3. Use it TOKEN="..." curl http://localhost:8000/api/v1/items -H "Authorization: Bearer $TOKEN"
```

A pre-baked Postman collection at `Backend/docs/InventoryBackend.postman_collection.json` automates this — import it, hit Auth → Login, and every other request inherits the token.

---

13. Known not-yet-built (Phase 9+)

- **Integration Layer wiring** — Zoho sync, webhook ingestion, idempotency keys. The `reference_type` / `reference_id` columns on `stock_movements` are designed to point at Integration-Layer-originated records when that phase lands.
- **Customer-specific pricing.** `customers.is_privileged` is reserved; the SO create flow uses the item's list price uniformly today.
- **Header-level discounts/taxes on PO/SO.** Schema reserves `total` as a separate column from `subtotal` for exactly this — adding it later is a service-level change, no migration.
- **Concurrency stress tests.** The `SELECT FOR UPDATE` correctness is pinned by code review and the atomicity test, not yet by a true concurrent-load harness.

---

14. Appendix — file map

```
Backend/ ■■■ src/app/ ■ ■■■ api/v1/ # routers, one per resource ■ ■ ■■■ auth.py ■ ■ ■■■ users.py ■ ■ ■■■ items.py ■ ■ ■■■ customers.py ■ ■ ■■■ vendors.py ■ ■ ■■■ vendor_terms.py ■ ■ ■■■ stock_movements.py ■ ■ ■■■ purchase_orders.py ■ ■ ■■■ sales_orders.py ■ ■■■ core/ # config, security, db, logging, exceptions ■ ■■■ dependencies/ # auth, db Depends() providers ■ ■■■ middleware/ # request-id, access log ■ ■■■ models/ # SQLAlchemy ORM (one file per table family) ■ ■■■ repositories/ # all SQL — services compose these ■ ■■■ schemas/ # Pydantic I/O models ■ ■■■ services/ # business logic ■ ■■■ main.py # app factory + router wiring ■■■ migrations/ # Alembic, runs on test gate + dev deploy ■■■ tests/ # 199 tests across 11 files ■■■ docs/ # this reference + Postman collection ■■■ README.md, CLAUDE.md # operator + contributor entrypoints
```

Generated as part of the Phase 1–8 Backend deliverable. Questions / errata: file an issue against the repo or ping the backend owner.